

Mitigating Covariate Shift in Imitation Learning for Autonomous Vehicles Using Latent Space Generative World Models

Alexander Popov, Alperen Degirmenci, David Wehr, Shashank Hegde, Ryan Oldja, Alexey Kamenev
Bertrand Douillard, David Nistér, Urs Muller, Ruchi Bhargava, Stan Birchfield, Nikolai Smolyanskiy
NVIDIA

Abstract—We propose the use of latent space generative world models to address the covariate shift problem in autonomous driving. A world model is a neural network capable of predicting an agent’s next state given past states and actions. By leveraging a world model during training, the driving policy effectively mitigates covariate shift without requiring an excessive amount of training data. During end-to-end training, our policy learns how to recover from errors by aligning with states observed in human demonstrations, so that at runtime it can recover from perturbations outside the training distribution. Additionally, we introduce a novel transformer-based perception encoder that employs multi-view cross-attention and a learned scene query. We present qualitative and quantitative results, demonstrating significant improvements upon prior state of the art in closed-loop testing in the CARLA simulator, as well as showing the ability to handle perturbations in both CARLA and NVIDIA’s DRIVE Sim.¹

I. INTRODUCTION

Autonomous vehicles must navigate complex and dynamic environments. In terms of both data and compute, the most effective method is to learn from human driving. Recent studies have trained neural planners using imitation learning (IL) [1]. However, IL is susceptible to the covariate shift problem [2], which impedes the development of effective driving policies.

Covariate shift occurs when the state distribution encountered by the planner’s policy during deployment differs from that during training. This discrepancy arises because the training data, which captures expert behavior, may not encompass all possible states the policy will face in practice. As a result, the driving policy may perform well on the training data but fail in new, unseen states when deployed. This results in the autonomous vehicle drifting away from optimal trajectories when guided by the trained neural planner—oftentimes with catastrophic effects.

One way to mitigate covariate shift is to collect recovery trajectories from bad states to good states via new human demonstrations. Another way is to augment data by stochastically sampling new states and then using a privileged planner in simulation to compute recovery trajectories [2], [3]. Unfortunately, such methods are fragile, typically use sub-optimal heuristics, and are expensive to implement.

In this paper, we demonstrate that co-training driving policies with generative latent space world models mitigates the covariate shift problem. A world model, as described in

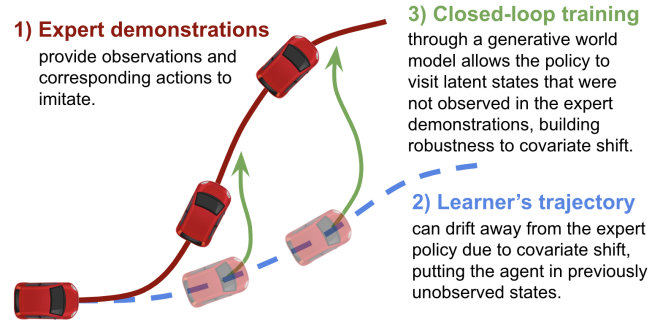


Fig. 1: Training with a latent generative world model allows our policy to visit latent states that were not observed in the expert demonstrations, and learn to recover by minimizing the KL-divergence between world model rollouts and expert demonstrations, teaching the policy to select actions that guide the agent toward favorable states.

[4], is a deep neural network (DNN) trained to represent the ego vehicle’s state, traffic dynamics, and the static structure of the environment. Such models are trained using expert demonstrations recorded from driving data (including sensor data, vehicle trajectories, and navigation goals) to predict future states based on past states and driving actions.

We train a latent space generative world model that allows us to sample new ego states from the learned latent space that were not present in the training data. These sampled states are then used to train the driving policy to recover from errors, where the policy learns to pick actions such that future latent states are closer to the states observed in human demonstrations (Fig. 1). Our experiments show that using latent space generative world models can solve or mitigate the covariate shift problem in imitation learning.

Our contributions are as follows:

- We train an end-to-end driving network with a latent space generative world model that addresses the covariate shift problem.
- We introduce a novel transformer-based perception encoder architecture (leveraging DINOv2 [5] as a featurizer) that facilitates the learning of effective world state representations used by the world model.
- We present qualitative and quantitative results of closed-loop driving in both the CARLA simulator [6] and the more realistic NVIDIA DRIVE Sim simulator.

¹Video is at <https://youtu.be/7m3bXz1VQvU>.

II. PREVIOUS WORK

A. Covariate Shift in Imitation Learning

Ross et al. [2] introduce DAgger that structures prediction problems as no-regret online learning. This approach ensures performance improvements over time by iteratively aggregating novel states where the agent may fail through a training-deployment-collection-training loop. Chang et al. [3] address the covariate shift challenge with limited offline data using model-based imitation learning (MILO). MILO learns an effective policy with partial expert action coverage, enhancing robustness in real-world applications. However, it assumes direct access to the world state, does not consider noisy sensor state estimation, uses a discrete MDP, and assumes calibrated transition function uncertainty. Spencer et al. [7] argue that the divergence between “held out” error and performance of the learner *in situ* is a manifestation of covariate shift, which can be mitigated by taking advantage of a simulator without further need for expert demonstrations. Ke et al. [8] address the covariate shift in model-free imitation learning for manipulation tasks like grasping with chopsticks. Their technique enhances the robustness of learned policies, leading to improved performance in real-world scenarios. Tennenholtz et al. [9] explore latent confounders’ covariate shift in both imitation and reinforcement learning (RL). The authors introduce techniques to detect and mitigate the impact of these confounders, enhancing the robustness of learned policies. In contrast to these approaches, we mitigate covariate shift by using the world model as a neural simulator to generate new states in the latent space during training.

B. World Models

Ha and Schmidhuber [4] introduced world model learning in 2018, training a model to forecast traffic in latent space, which could be decoded into Cartesian top-down space. They combined RNN and VAE to predict both top-down driving episodes and perspective views. The trained control policies using these world models allowed the policy to explore various scenarios and handle general cases as well as recover from unusual or adverse states. Dreamer [10] learns a latent space world model from a replay buffer of past experiences. It uses actor-critic RL and a learned world model to train a control policy. This system can learn long-horizon behaviors solely from images and has demonstrated state-of-the-art performance on various control tasks in the ATARI domain. DayDreamer [11] is an extension of Dreamer applied to physical robot learning in the real world, without using simulators. MILE [12] adapted the Dreamer approach to the AV domain and tested it in the CARLA simulator [6]. It learns a latent-space generative world model from expert driving demonstrations and uses it to train an end-to-end driving policy. Unlike Dreamer, it does not use RL in training and focuses on imitation learning. MILE can imagine future driving scenarios based on different ego actions, which are decoded as bird’s-eye views (BEV). The system relies on LSS [13] as its perception encoder. PredictionNet [14] is an auto-regressive RNN-based world model that predicts

future vehicle states, occupancies, and traffic flow for all traffic agents, including the ego. It can be used for ego action conditioned traffic prediction. Unlike Dreamer [10], it operates in a BEV space.

Our latent-space generative world model approach is similar to JEPA [15], Dreamer [10], and MILE [12]. JEPA was only used for visual tasks, not vehicle planning and control. Unlike Dreamer, which employs a critic and RL for training, we derive our driving policy from human expert driving demonstrations, similar to MILE. Unlike MILE, we introduce a novel multi-view transformer-based perception encoder, utilize DINOv2 [5] as an image featurizer, use different training recipe and investigate covariate shift.

C. Planning and Control

Several works proposed end-to-end approaches for AV using deep neural networks, including ChauffeurNet [16], PlaNet [17], and UniAD [18]. These neural planners may encounter covariate shift, making it crucial to address this issue. ChauffeurNet mitigated it by introducing small random ego pose augmentations during DNN training. While this approach is similar to the concept in DAgger [2], the specific set of augmentations is heuristically created, which is challenging to implement.

III. METHOD

A. System Architecture

Fig. 2 illustrates the overall architecture of our system, comprising several DNNs co-trained end-to-end. Multilayer perceptrons (MLP) are used throughout the system for tensor shape adaptation between DNNs.

Intuitively, the architecture can be compared to behavior cloning (BC), where the perception encoder maps the input to an observation feature vector, which the driving policy uses to generate the next action. BC faces the covariate shift problem, since the policy is trained to follow the expert without being exposed to deviations. BC can be improved by tracking the state over time using a history network and state estimator, with backpropagation over the entire episode to capture long-term dependencies. However, covariate shift remains an issue when the system encounters unseen states.

To address covariate shift, our system uses a latent space generative world model that models state transitions conditioned on ego action. This transforms behavior cloning into a temporal world model-based imitation learning system. During backpropagation, gradients flow through the driving policy and world model, affecting previous actions over multiple time-steps, enabling recovery from mistakes. The recovery signal comes from the difference between latent states predicted by the world model and those computed by the state estimator. The system is trained end-to-end, with different networks converging at different rates, and the policy network converging last.

Why does this work? The world model acts as a latent-space neural simulator that samples novel states at training time, exposing the system to these novel states to allow the

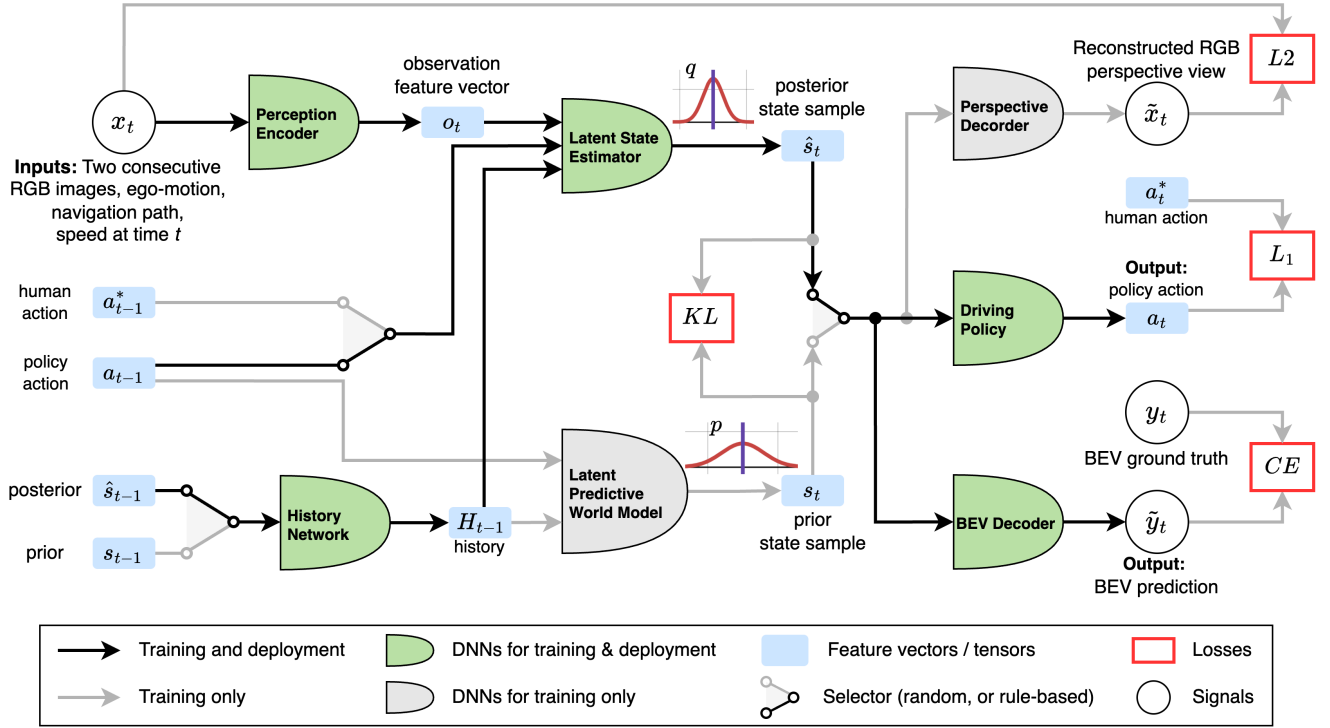


Fig. 2: System architecture diagram showing transition from time $t - 1$ to t . During training we backpropagate through an entire episode consisting of 12 timesteps (red boxes show the losses). During deployment the system is operated recurrently.

policy to learn how to properly handle them. Since high-dimensional scene data are mapped to a lower-dimensional latent space with a smooth manifold, the trained policy is able to recover from rogue states (that is, move back toward states on the latent world model manifold that more closely resemble those associated with the expert demonstrations).

B. System component details

Perception encoder illustrated in Fig. 3 takes as input two sequential RGB frames (at 10 Hz) from the monocular front camera, corresponding past ego-motion (6 DOF vehicle pose transformation), the desired upcoming navigation path (192×192 binary bird’s-eye views (BEV) image, 0.2 m per pixel resolution), and the current vehicle speed. Each RGB image is featurized by a DINOv2 [5] backbone to produce a set of image tokens F_t . Motion information is incorporated by a cross-attention layer [19] between queries F_t , keys, and values F_{t-1} to produce a set of tokens M_t that represent the currently observed scene. Camera intrinsics and extrinsics are processed by MLPs and added to the image features prior to cross-attention to help with multi-view matching similar to [20]. M_t is then processed via 4 blocks of self-attention with 8 heads into tokens S_t . Finally, a learned query Q (similar to [21]) is used to cross-attend with keys and values to output a single vector to represent the observation feature vector $o_t \in \mathbb{R}^{512}$. The desired navigation path and ego speed are embedded via MLPs and then added to the scene embedding. This scene observation feature vector o_t represents currently observed scene and is passed to the

state estimator. Observation feature vector o_t can also be concatenated with GPS and IMU data encoded as 1D vectors.

We use DINOv2 pre-trained backbone [5] as an image featurizer. This allows us to train a robust perception encoder on real data that can be deployed in a simulator for validation, without requiring fine-tuning to close the sim-to-real gap. If validation in simulation is successful, the trained end-to-end driving stack can be directly tested on a real vehicle, as it has been exclusively trained on real data.

History network aggregates previous states deterministically using the hidden state of a GRU RNN. We employ the same GRU settings as in DreamerV3 [10].

State estimator models the latent state posterior distribution conditioned on observations. It estimates a surrogate distribution $q(\hat{s}_t|o_t, H_{t-1}, \hat{a}_{t-1})$ as a Gaussian, that is referred to as the posterior. It approximates the true hidden state using evidence lower bound (ELBO) [22]. This generative DNN takes observed perception feature vector o_t , past history H_{t-1} (which incorporates past state samples up to and including time $t - 1$), featurized human expert ego action $a_{t-1}^* \in \mathbb{R}^2$ at time step $t - 1$. Expert ego action embedding is done by passing the combined steering and acceleration values via MLP. The state estimator computes the parameters of a Gaussian distribution, from which we then sample posterior latent states $\hat{s}_t \in \mathbb{R}^{512}$. This DNN is implemented as an MLP, as in DreamerV3 [10]. The state samples are recursively used for the next iterations via the history network.

World model predicts stochastic latent space world tran-

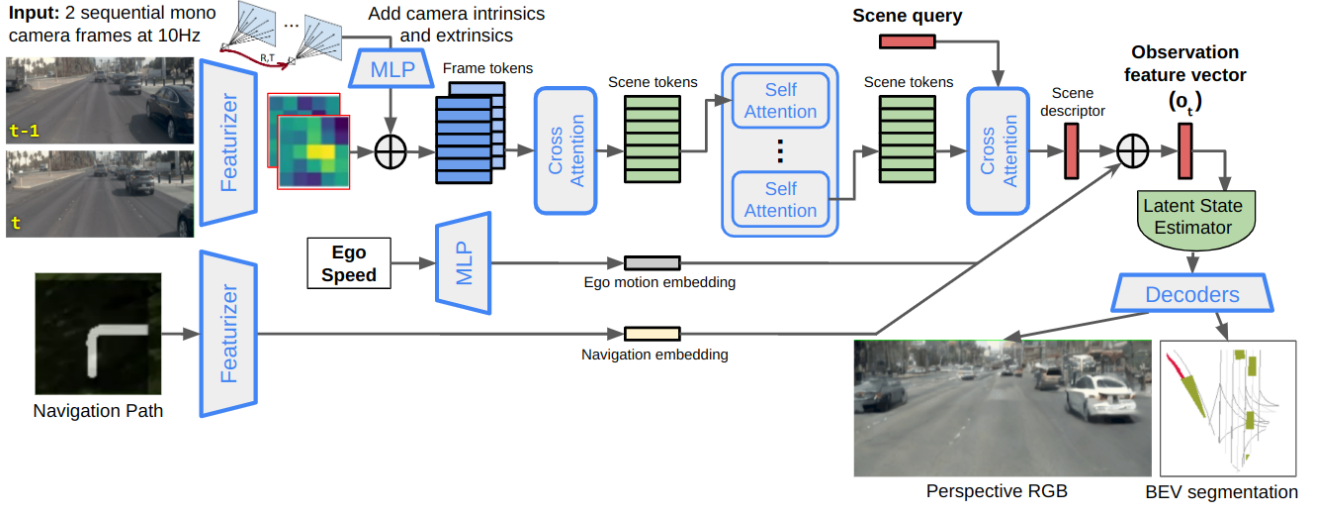


Fig. 3: Our novel transformer-based multi-view perception encoder. Input data is tokenized, and frame tokens are cross-attended to match patches. Scene descriptor is computed via cross-attention between learned scene query and scene tokens. Output is computed by adding navigation and ego motion embeddings to the scene descriptor.

sitions from s_{t-1} to $s_t \in \mathbb{R}^{512}$ conditioned on ego action and history at time $t-1$. It takes as input the past history H_{t-1} , the ego action $a_{t-1} \in \mathbb{R}^2$ produced by the driving policy for time step $t-1$. The world predictor is a generative DNN that predicts the latent state as a Gaussian distribution, from which we sample the s_t prior state $p(s_t | H_{t-1}, a_{t-1})$. The DNN is implemented as an MLP, similar to DreamerV3 [10]. This DNN can be used to roll out new “imagined” states without corresponding observations up to a given horizon. We can use these roll-outs for re-simulating long-tail situations. The latent predictive world model is not run at deployment time.

Driving policy network is an MLP network with four layers and ReLU activations. Input dimensions are the same as latent state dimensions and output dimension is two, corresponding to steering and acceleration/braking actions.

We iterate the system over time by using either s_t or $\hat{s}_t$ for the next iteration with some probability. Switching between s_t and $\hat{s}_t$ helps mitigate covariate shift problems. Since s_t is sampled stochastically, it may include unusual states from which the model is trained to recover. This can be seen as a form of exploration (as in RL).

The decoder DNNs decode latent state samples $s_{(\cdot)}$ into perspective RGB views and bird’s-eye views (BEV) semantic segmentation views. We experimented using GANs (StyleGAN2 [23]) or latent diffusion model based on VideoLDM [24] [25] with a pre-trained VAE from [26], which worked much better than GANs. The decoders are trained using reconstruction losses against input sensor data (RGB frames) and top-down BEV ground truth targets. This allows decoding a sequence of latent states into a temporally cohesive perspective RGB sequence, which is useful for system inspection and visualization. The introspection capabilities provided by these decoders is critical for safety checks in end-to-end trained autonomy stacks.

C. Training and Losses

We use variational inference approach to train the system by maximizing the ELBO [22].² Our objective function is $\mathcal{L} = \mathcal{L}_{Recon} + \mathcal{L}_{KL}$, similar to [12], where

$$\mathcal{L}_{Recon} = \sum_{t=1}^T \mathbb{E}_p \left[\log p(x_t | H_{t-1}, s_t) + \log p(y_t | H_{t-1}, s_t) + \log p(a_t | H_{t-1}, s_t) \right] \quad (1)$$

$$\mathcal{L}_{KL} = - \sum_{t=1}^T \mathbb{E}_q [D_{KL}(q(\hat{s}_t | o_t, H_{t-1}, \hat{a}_{t-1}) || p(s_t | H_{t-1}, a_{t-1}))] \quad (2)$$

where x_t is the reconstructed RGB image, y_t is the reconstructed BEV semantic label image, and a_t is the action. $p(x_t, \cdot)$ follows a Gaussian distribution, leading to L2 loss; $p(y_t, \cdot)$ follows a categorical distribution, leading to cross entropy loss; and $p(a_t, \cdot)$ follows a Laplace distribution, leading to L1 loss.

All DNN modules in our system were trained simultaneously by optimizing the objective $\mathcal{L}$. We have observed empirically that the training process proceeds in the following way: the perception encoder and decoder converge first, followed by the world predictor, and finally the driving policy.

IV. EXPERIMENTS

We conducted several sets of experiments to evaluate our system.

²Note that our ELBO-based loss is the same as the free-energy principle objective used in active inference within cognitive science [27].

TABLE I: Results on CARLA closed-loop simulator [6] on held-out test data and new weather conditions using the same evaluation procedure as [12]. See text for details.

Method	Single Cam	LiDAR	Driving Score $\uparrow$	Route $\uparrow$	Infraction $\uparrow$
CILRS [28]	yes $\checkmark$	no $\checkmark$	7.8 ± 0.3	10.3 ± 0.0	76.2 ± 0.5
LBC [29]	yes $\checkmark$	no $\checkmark$	12.3 ± 2.0	31.9 ± 2.2	66.0 ± 1.7
TransFuser [30]	no $\times$	yes $\times$	31.0 ± 3.6	47.5 ± 5.3	76.8 ± 3.9
Roach [31]	yes $\checkmark$	no $\checkmark$	41.6 ± 1.8	96.4 ± 2.1	43.3 ± 2.8
LAV [32]	no $\times$	yes $\times$	46.5 ± 3.0	69.8 ± 2.3	73.4 ± 2.2
TCP [33]	yes $\checkmark$	no $\checkmark$	57.0 ± 1.9	85.3 ± 1.2	67.0 ± 1.0
MILE [12]	yes $\checkmark$	no $\checkmark$	61.1 ± 3.2	97.4 ± 0.8	63.0 ± 3.0
InterFuser* [34]	no $\times$	yes $\times$	68.3 ± 1.9	95.0 ± 2.9	—
TF++ [35]	yes $\checkmark$	yes $\times$	70.0 ± 6.0	99.0 ± 0.0	70.0 ± 6.0
ReasonNet* [36]	no $\times$	yes $\times$	73.2 ± 1.9	95.9 ± 2.3	76.0 ± 3.0
Ours	yes $\checkmark$	no $\checkmark$	70.0 ± 0.2	100.0 ± 0.1	80.0 ± 0.7
Expert (RL agent with privileged info) [31]			88.4 ± 0.9	97.6 ± 1.2	90.5 ± 1.2

A. CARLA Simulator Closed-loop Evaluation

For this experiment, we trained our system using data from the CARLA simulator [6]. We collected expert driving data in CARLA using a pre-trained agent (trained via reinforcement learning) following MILE [12]. We gathered 50k episodes of driving, each 1.2 seconds long and consisting of 12 frames sampled at 10 Hz (to allow sufficient separation between frames). Our CARLA training dataset was collected in the CARLA simulator, similar to [12], using a privileged RL-based expert driver [31]. We then jointly trained all the networks of our system end-to-end on this dataset. The training process spanned 15 million episodes, with an initial learning rate of 10^{-4} and an effective batch size of 64, utilizing the *1cycle* [37] learning rate schedule. The total training time was 5 days on 16 NVIDIA A100 GPUs.

Our trained system was deployed in the closed-loop CARLA simulator [6] for evaluation. Our system successfully reacted to traffic lights, followed lead vehicles, stopped for them, avoided collisions, allowed pedestrians to pass, and adhered to a given route. All training and inference were performed end-to-end without any handcrafted code. We quantitatively evaluated our system by computing closed-loop metrics according to the CARLA Leaderboard 1.0 (Sensors Track).³ Following MILE [12], we evaluated on held-out Town05 and with new weather conditions, which were excluded from training.

The results, averaged over three runs, are shown in Table I, where the comparative baselines are taken from the table in MILE [12]. (To ensure a fair comparison, we reproduced the numbers from MILE itself.) Our table also includes four methods that were published after MILE. For two of these methods (TCP [33] and TF++ [35]), we grabbed the numbers from their respective papers; these results are shown in italics. We also collected numbers from the tables in two additional papers (InterFuser [34] and ReasonNet [36]) that

show results on Town05 Long, which is the same held-out test map but without new weather conditions; these results are indicated with italics and an asterisk.

Our approach achieves better CARLA evaluation scores than prior single-camera state-of-the-art, and it even outperforms many methods that use multiple cameras and/or LiDAR. Note that our system was trained on just 50k episodes, compared with 242k episodes for MILE [12].

B. Covariate Shift Experiments

To test whether training with latent space generative models mitigate covariate shift, we compared our system against a behavior cloning (BC) method to imitate human driver actions. In this ablation study, we kept the same BC architecture as in Fig. 2 during inference but made two changes during training: 1) The world model network (along with KL-divergence) was removed, and 2) Instead of stochastically sampling the posterior distribution of the state estimator, the deterministic mean was used. These changes result in a BC system with state tracking over time (by history network).

We compared the two approaches in the closed-loop CARLA simulator using Town02. Results shown in Table II demonstrate successful navigation of our system without drifting into undesirable states. In contrast, the policy trained via behavior cloning (BC) achieves worse scores across multiple metrics in the same simulator.⁴ The significant difference demonstrates the effectiveness of our latent space generative world model in mitigating covariate shift. Note, however, that our BC implementation still achieves good performance, primarily due to the robust visual features obtained by DINOv2.

C. Disturbance Experiments

As a continuation of the ablation study, we also conducted qualitative disturbance experiments in the closed-loop CARLA simulator [6]. Disturbances were introduced

³Like MILE [12], we were unable to submit to the online leaderboard, <https://leaderboard.carla.org/leaderboard> which is now closed. CARLA leaderboard 2.0 was not possible, since it would require uploading our code; and it contains few methods.

⁴Adding stochastic sampling to BC improves results slightly, but still leads to drifting and collisions. In our experience, training BC on an order of magnitude more data does not resolve covariate shift.

TABLE II: Results of our system vs. behavior cloning (BC) for Town02 only.

Metric	GenWorldModel (ours)	Behavior Cloning (BC)
Route comp., no crash $\uparrow$	80.0%	30.0%
Route comp., km $\uparrow$	0.9	0.8
Driving Score $\uparrow$	80.0	70.0
Reward $\uparrow$	3500.4	2936.8

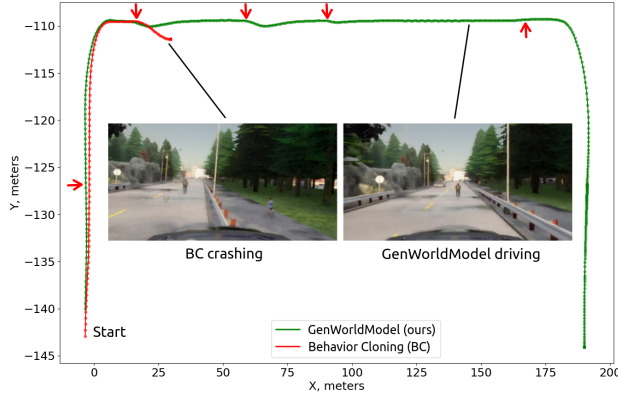


Fig. 4: Top-down qualitative comparison of policies driving along a route in CARLA [6], while steering control disturbances were applied (red arrows). Behavior cloning was not able to recover from the second disturbance, hitting the guardrail and terminating early. Our system successfully recovered and completed the route.

by manually overriding the driving policy for 300 ms at a time (3 consecutive frames at 10 Hz) with a 30° steering command to turn either right or left, then allowing the driving policy to resume control.

Fig. 4 shows an example of our method versus the behavior cloning (BC) policy described above, on a route involving two 90-degree right turns. The BC policy was unable to recover from the second disturbance, causing the vehicle to crash into the guardrail. Our policy, on the other hand, successfully navigated the route despite the many disturbances. In other examples, we observed that our driving policy was able to recover to normal driving, even when the car was pushed onto the sidewalk or into oncoming traffic.

D. DRIVE Sim Experiments

We also tested our model in NVIDIA DRIVE Sim, which is more photo-realistic and physically realistic than CARLA (Fig. 5). We trained our system using a 450-hour internal real driving dataset collected by multiple vehicles in our autonomous vehicle fleet in several geographic locations, consisting of 1.2-second expert driving episodes with 12 frames sampled at 10 Hz, amounting to 1.3 million episodes. The training process was the same as that used in the CARLA experiments. The system trained for 15 days on 64 NVIDIA A100 GPUs. We then deployed the system in closed-loop

NVIDIA DRIVE Sim, evaluating its performance across various scenarios and testing its recovery ability by artificially introducing a control lag of approximately 250 ms. The driving policy demonstrated robust performance and was able to recover effectively. Qualitative results can be seen in supplemental video material. Our model runs in near real time (10Hz) on NVIDIA Orin AGX embedded GPU.

E. Perspective View RGB Decoding

Recall from Eq. (1) that the system reconstructs the RGB perspective image x_t from the latent space. While StyleGAN decoder works well for simulated image data as shown in [12], we found that the generated images are of poor quality and not temporally consistent when applied to more complex real data from the nuPlan dataset [38].

In contrast, our latent diffusion video decoder is able to generate high quality, temporally consistent 832×320 RGB frames that match the general scene setup of the original image frames used to initialize episodes. A typical reconstructed image is shown in Fig. 6.

V. CONCLUSION

In this work, we have shown that a driving policy trained with a latent generative world model mitigates the imitation learning covariate shift problem. We introduced a novel multi-view transformer-based perception encoder using self-supervised pre-training and DINOv2. We showed qualitative and quantitative results on CARLA Sim, and qualitative results on NVIDIA DRIVE Sim. Future work includes enhancing the world model predictor by adopting architectures based selective state space models like Mamba [39], investigating whether training with the world model reduces the amount of data required for neural policies, and exploring the use of reinforcement learning to handle long-tail scenarios.

ACKNOWLEDGMENT

We thank Arthur Przech, Xiaolin Lin, George Hines, Ephrem Chemali, Ankit Gupta, Seth Robert Piezas, Kayley Ting, Hai Loc Lu, Ryan Holben and Yashraj Narang for contributions and feedback.

REFERENCES

- [1] A. G. Barto and R. S. Sutton, "Learning from delayed rewards," *Cognitive Science*, vol. 7, no. 3, pp. 329–354, 1983. 1
- [2] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635. 1, 2
- [3] J. Chang, M. Uehara, D. Sreenivas, R. Kidambi, and W. Sun, "Mitigating covariate shift in imitation learning via offline data with partial coverage," *Advances in Neural Information Processing Systems*, vol. 34, pp. 965–979, 2021. 1, 2
- [4] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018. 1, 2
- [5] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "Dinov2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023. 1, 2, 3



Fig. 5: Our end-to-end system driving in the CARLA (top) and NVIDIA DRIVE Sim (bottom) simulators.



Fig. 6: Original image (top) and reconstructed image (bottom) from our latent diffusion-based decoder. The latent representation faithfully captures much of the scene, although artifacts are visible (vehicles on the left).

[6] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” in *Conference on Robot Learning (CoRL)*, 2017. 1, 2, 5, 6

[7] J. Spencer, S. Choudhury, A. Venkatraman, B. Ziebart, and J. A. Bagnell, “Feedback in imitation learning: the three regimes of covariate shift,” *arXiv preprint arXiv:2102.02872*, 2021. 2

[8] L. Ke, J. Wang, T. Bhattacharjee, B. Boots, and S. Srinivasa, “Grasping with chopsticks: Combating covariate shift in model-free imitation learning for fine manipulation,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 6185–6191. 2

[9] G. Tennenholtz, A. Hallak, G. Dalal, S. Mannor, G. Chechik, and U. Shalit, “On covariate shift of latent confounders in imitation and reinforcement learning,” in *International Conference on Learning Representations*, 2022. 2

[10] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” in *International Conference on Learning Representations*, 2020. 2, 3, 4

[11] P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg, “Daydreamer: World models for physical robot learning,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, K. Liu, D. Kulic, and J. Ichnowski, Eds., vol. 205.

PMLR, 14–18 Dec 2023, pp. 2226–2240. 2

[12] A. Hu, G. Corrado, N. Griffiths, Z. Murez, C. Gurau, H. Yeo, A. Kendall, R. Cipolla, and J. Shotton, “Model-based imitation learning for urban driving,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 20703–20716. 2, 4, 5, 6

[13] J. Philion and S. Fidler, “Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3D,” in *ECCV*, A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, Eds., 2020. 2

[14] A. Kamenev, L. Wang, O. B. Bohan, I. Kulkarni, B. Kartal, A. Molchanov, S. Birchfield, D. Nistér, and N. Smolyanskiy, “PredictionNet: Real-time joint probabilistic traffic prediction for planning, control, and simulation,” in *International Conference on Robotics and Automation (ICRA)*, 2022, pp. 8936–8942. 2

[15] Y. LeCun, “Joint embedding predictive architecture (jepa),” *arXiv preprint arXiv:2207.00000*, 2022. 2

[16] M. Bansal, A. Krizhevsky, and A. Ogale, “ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst,” in *Robotics Science and Systems (RSS)*, 2019. 2

[17] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, and H. Lee, “Learning latent dynamics for planning from pixels,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97, 2019, pp. 2554–2563. 2

[18] Y. Hu *et al.*, “Planning-oriented autonomous driving,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” vol. 30, 2017. 3

[20] A. Zisserman *et al.*, “Input-level inductive biases for 3d reconstruction,” *arXiv preprint arXiv:2112.03243*, 2021. 3

[21] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” *arXiv preprint arXiv:2005.12872*, 2020. 3

[22] D. M. Blei, A. Kucukelbir, and J. D. McAuliffe, “Variational inference: A review for statisticians,” *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, 2017. 3, 4

[23] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of stylegan,” *arXiv preprint arXiv:1912.04958*, 2020. 4

[24] A. Blattmann, R. Rombach, H. Ling, T. Dockhorn, S. W. Kim, S. Fidler, and K. Kreis, “Align your latents: High-resolution video synthesis with latent diffusion models,” *arXiv preprint arXiv:2304.08818*, 2023. 4

[25] T. Karras, M. Aittala, T. Aila, and S. Laine, “Elucidating the design space of diffusion-based generative models,” in *Proc. NeurIPS*, 2022. 4

[26] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts, *et al.*, “Stable video diffusion: Scaling latent video diffusion models to large datasets,” *arXiv preprint arXiv:2311.15127*, 2023. 4

[27] K. Friston, “The free-energy principle: a unified brain theory?” *Nature Reviews Neuroscience*, vol. 11, no. 2, pp. 127–138, 2010. 4

- [28] F. Codevilla, E. Santana, A. M. López, and A. Gaidon, “Exploring the limitations of behavior cloning for autonomous driving,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9329–9338. 5
- [29] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, “Learning by cheating,” in *Conference on Robot Learning*, 2020, pp. 66–75. 5
- [30] A. Prakash, K. Chitta, and A. Geiger, “Multi-modal fusion transformer for end-to-end autonomous driving,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 7077–7087. 5
- [31] Z. Zhang, A. Liniger, D. Dai, F. Yu, and L. V. Gool, “End-to-end urban driving by imitating a reinforcement learning coach,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2021, pp. 15 222–15 232. 5
- [32] D. Chen and P. Krähenbühl, “Learning from all vehicles,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 5
- [33] P. Wu, X. Jia, L. Chen, J. Yan, H. Li, and Y. Qiao, “Trajectory-guided control prediction for end-to-end autonomous driving: A simple yet strong baseline,” in *NeurIPS*, 2022. 5
- [34] H. Shao, L. Wang, R. Chen, H. Li, and Y. Liu, “Safety-enhanced autonomous driving using interpretable sensor fusion transformer,” in *CoRL*, 2022. 5
- [35] B. Jaeger, K. Chitta, and A. Geiger, “Hidden biases of end-to-end driving models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. 5
- [36] H. Shao, L. Wang, R. Chen, S. L. Waslander, H. Li, and Y. Liu, “Reasonnet: End-to-end driving with temporal and global reasoning,” in *CVPR*, 2023. 5
- [37] L. N. Smith, “Super-convergence: Very fast training of neural networks using large learning rates,” *arXiv preprint arXiv:1803.09820*, 2018. 5
- [38] H. Caesar, J. Kabzan, K. Tan, *et al.*, “NuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles,” in *CVPR ADP3 workshop*, 2021. 6
- [39] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023. 6